Conditionals

- **break statement**
  - Exit a loop immediately
  - Used also to escape switch statements
- **continue**
  - Skip the current current of the loop and move to the next
- **goto**
  - Unconditional jump to a block of code.
  - Used to get out of deeply nested loops

Loops

- **Entry Controlled**
  - Loops that are executed only if a certain condition is met
  - Condition is checked first, then body is executed.
  - Example: for, while.
- **Exit controlled**
  - do-while, executes atleast
- Loop variable default value is undefined.
- Function calls are valid inside the initialization part of the loop

Arrays

- Arrays need to have their size declared at creation so compiler can allocate memory to it

→ You can skip mentioning the size of the array if you initialize the array at the same time.

```
int arr[] = {1, 2, 3};
```

↳ array of size 3.

→ Also, if you initialize an array but only populate half of it, the rest will get populated by 0's

→ Strings do not exist, so we have arrays of characters instd.

```
char arr[MAX_STRINGS][STRING_LENGTH+1]
```

Initialization can be done using strcpy

```
arr[5][10] = {"Hello", "World"};
```

→ now in the case that you wanted more dynamic and increasing length of strings, you would need to use malloc. and using a a buffer of strlen + 1 and using strcpy on that

```
char *arr[NO_OF_STRINGS] = {"Hello", "World"};
```

↑

Also can use this since were only storing pointers

But to edit it, you would need to do

```
arr[i] = malloc(strlen("whatever your string" + 1));

strcpy(arr[i], "Whatever your string");

free(arr[i]);
```

(CAUTION: malloc and free are stdlib.h functions)

→ scanf needs an "&" operator as C only has pass-by-value parameters. This means to pass a value into a variable we need to pass its address or a pointer.

→ When passing an array, the pointer of the array is passed, meaning I dont have access to the full array. Meaning I need to send the size of the array as an argument as well.

Reverse an Array

```
void rev(int arr[], size_t size)
{
    int j, k;
    j = 0; k = size - 1;

    for(; (size%2==0) ? j == size/2 : j==size/2-1;)
```

1 2 3 4 5 6 7 8 9
i . . . . j
j == size/2 - 1

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
. . . .
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

```
for ( i = size - 1;  i > index ; i -- )
  {
```